

Week 5 Structured Notes: Advanced Recursive Problems

1. Week 5 Main Goal

The goal of Week 5 is to move from simple recursive problems to more DSA-style recursive problems.

In Week 4, you learned problems like:

- `sum(n)`
- `factorial(n)`
- `power(m, n)`
- `fastPower(m, n)`

Those problems taught this pattern:

```
return smaller_problem + or * current_work;
```

Week 5 goes further. It teaches that a recursive solution can be correct but still inefficient.

By the end of Week 5, you should understand:

- Taylor series using recursion
- Horner's Rule
- Fibonacci using loop
- Fibonacci using basic recursion
- Why basic Fibonacci is slow
- Fibonacci using memoization
- Why `-1` is used in memoization
- `nCr` using the factorial formula
- `nCr` using Pascal's Triangle recursion
- Repeated recursive work
- Tree recursion in real problems
- Basic optimization using stored results

The main Week 5 question is:

Does this recursive solution only work, or does it work efficiently?

2. Quick Revision Before Week 5

Before learning Week 5, remember these ideas from earlier weeks.

2.1 Recursion

Recursion means a function calls itself.

Example:

```
void fun(int n) {  
    if (n > 0) {  
        fun(n - 1);  
    }  
}
```

Here, `fun()` calls itself, so it is recursive.

2.2 Base Case

The base case stops recursion.

Example:

```
if (n == 0) {  
    return 1;  
}
```

Without a base case, recursion may continue forever and cause stack overflow.

2.3 Recursive Case

The recursive case is where the function calls itself again.

Example:

```
return factorial(n - 1) * n;
```

Here, `factorial(n - 1)` is the recursive call.

2.4 Stack Memory

Each recursive call creates a new stack frame.

Example:

```
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

Each call waits on the stack until the smaller call finishes.

2.5 Tree Recursion

Tree recursion happens when a function calls itself more than once.

Example:

```
return fib(n - 1) + fib(n - 2);
```

This creates two recursive branches.

Tree recursion can become slow because many calls may repeat the same work.

3. Why Week 5 Is Important

Week 5 is important because it teaches the difference between:

```
A solution that works
```

and

```
A solution that works efficiently
```

For small inputs, inefficient recursion may look fine.

But for large inputs, repeated recursive calls can become very slow.

Example:

```
fibRecursive(5)
```

is small and easy.

But:

```
fibRecursive(40)
```

can become very slow because the same values are calculated again and again.

This is why Week 5 introduces memoization.

Memoization means storing answers after calculating them once.

4. Day 1: Taylor Series Using Recursion

4.1 What Is Taylor Series?

In this recursion topic, Taylor series is used to calculate an approximate value of:

```
e^x
```

The formula is:

$$e^x = 1 + x/1! + x^2/2! + x^3/3! + \dots + x^n/n!$$

This means the answer is built by adding many small terms.

Example for `e^1` up to 3 terms:

$$e^1 \approx 1 + 1/1! + 1^2/2! + 1^3/3!$$

Now simplify:

$$e^1 \approx 1 + 1 + 1/2 + 1/6$$

So:

$$e^1 \approx 2.6667$$

The more terms we add, the closer the answer gets to the real value.

4.2 Meaning of One Taylor Series Term

A general term looks like this:

$$x^n / n!$$

For example:

$$x^3 / 3!$$

means:

$$(x * x * x) / (3 * 2 * 1)$$

So each term has two main parts:

Part	Meaning
<code>x^n</code>	Power or numerator
<code>n!</code>	Factorial or denominator

The final answer is the sum of all terms.

4.3 What We Track in Recursive Taylor Series

To calculate Taylor series recursively, we usually track:

Item	Meaning
<code>power</code>	Stores powers like <code>x</code> , <code>x^2</code> , <code>x^3</code>
<code>fact</code>	Stores factorial values like <code>1!</code> , <code>2!</code> , <code>3!</code>
<code>result</code> or <code>sum</code>	Stores the total value of the series

4.4 Recursive Taylor Series Code

```
#include <stdio.h>

// Calculates e^x using Taylor series up to n terms
double taylor(int x, int n) {
    static double power = 1;
    static double fact = 1;

    if (n == 0) {
        return 1;
    }

    double result = taylor(x, n - 1);

    power = power * x;
    fact = fact * n;

    return result + power / fact;
}

int main() {
    printf("%lf\n", taylor(1, 3));
    return 0;
}
```

4.5 Base Case in Taylor Series

The base case is:

```
if (n == 0) {
    return 1;
}
```

Why return ?

Because the first term of Taylor series is:

So when reaches , the function returns the first term.

4.6 Recursive Case in Taylor Series

The recursive case is:

```
double result = taylor(x, n - 1);
```

This means:

```
First calculate the smaller Taylor series.  
Then add the current term.
```

The current term is:

```
power / fact
```

So the final line is:

```
return result + power / fact;
```

4.7 Dry Run of `taylor(1, 3)`

Function call:

```
taylor(1, 3)
```

Calling Phase

The function first goes down:

```
taylor(1, 3)  
taylor(1, 2)  
taylor(1, 1)  
taylor(1, 0)
```

Base case:

```
taylor(1, 0) returns 1
```

Returning Phase

Now the function calculates terms while returning.

For `n = 1`:

```
power = 1 * 1 = 1  
fact = 1 * 1 = 1  
term = 1 / 1 = 1  
result = 1 + 1 = 2
```

For `n = 2`:

```
power = 1 * 1 = 1  
fact = 1 * 2 = 2  
term = 1 / 2 = 0.5  
result = 2 + 0.5 = 2.5
```

For `n = 3`:

```
power = 1 * 1 = 1  
fact = 2 * 3 = 6  
term = 1 / 6 = 0.1667  
result = 2.5 + 0.1667 = 2.6667
```

Final answer:

```
2.6667
```

4.8 Important Warning About `static` Variables

In the Taylor code, these variables are static:

```
static double power = 1;  
static double fact = 1;
```


A static variable is created only once.

That means all recursive calls share the same `power` and `fact`.

This works for one calculation, but it can cause problems if the function is called multiple times.

Example:

```
printf("%lf\n", taylor(1, 3));  
printf("%lf\n", taylor(1, 3));
```

The second call may give a wrong answer because `power` and `fact` may still remember old values.

Simple rule:

Static variables are useful for learning this example, but use them carefully.

4.9 Time and Space Complexity of Taylor Series

For the recursive Taylor function:

Complexity	Value	Reason
Time	<code>O(n)</code>	The function makes one recursive call for each value from <code>n</code> to <code>0</code>
Space	<code>O(n)</code>	Each recursive call uses stack memory

5. Day 2: Horner's Rule

5.1 Why Do We Need Horner's Rule?

Normal Taylor series can involve repeated calculation of powers and factorials.

For example:

```
1 + x/1! + x^2/2! + x^3/3! + x^4/4!
```

If we calculate powers and factorials again and again, we do extra work.

Horner's Rule reduces unnecessary multiplication.

5.2 Main Idea of Horner's Rule

Instead of writing Taylor series like this:

$$1 + x/1! + x^2/2! + x^3/3!$$

We rewrite it in a nested form:

$$1 + x/1 * (1 + x/2 * (1 + x/3))$$

This nested form helps us reuse previous results.

5.3 Simple Meaning

Normal Taylor thinking:

Calculate each term separately.
Then add all terms.

Horner thinking:

Start from the inside.
Use the previous answer to build the next answer.

This avoids repeated multiplication.

5.4 Horner Recursive Code

```
#include <stdio.h>

// Helper function for Horner's Rule
double hornerHelper(double x, int n, double sum) {
    if (n == 0) {
        return sum;
    }

    sum = 1 + (x * sum) / n;
```

```

        return hornerHelper(x, n - 1, sum);
    }

    // Starts Horner's Rule with sum = 1
    double horner(double x, int n) {
        return hornerHelper(x, n, 1);
    }

    int main() {
        printf("%lf\n", horner(1, 3));
        return 0;
    }

```

5.5 Base Case in Horner's Rule

The base case is:

```

if (n == 0) {
    return sum;
}

```

This means:

When there are no more terms to process, return the final sum.

5.6 Recursive Case in Horner's Rule

The recursive case is:

```

sum = 1 + (x * sum) / n;
return hornerHelper(x, n - 1, sum);

```

This updates the sum and moves to a smaller value of `n`.

5.7 Dry Run of `horner(1, 3)`

Start:

```
sum = 1  
n = 3
```

Step 1:

```
sum = 1 + (1 * 1) / 3  
sum = 1.3333
```

Step 2:

```
n = 2  
sum = 1 + (1 * 1.3333) / 2  
sum = 1.6667
```

Step 3:

```
n = 1  
sum = 1 + (1 * 1.6667) / 1  
sum = 2.6667
```

Step 4:

```
n = 0  
return 2.6667
```

Final answer:

```
2.6667
```

5.8 Why Horner's Rule Is Faster

Horner's Rule is faster because it does not calculate every power and factorial separately.

It reuses the current `sum` to calculate the next value.

Simple comparison:

Method	Idea	Efficiency
Basic Taylor	Build terms directly	Can involve repeated work
Horner's Rule	Use nested form	Reduces unnecessary multiplication

5.9 Time and Space Complexity of Horner's Rule

Complexity	Value	Reason
Time	$O(n)$	One recursive call per term
Space	$O(n)$	Recursive calls use stack memory

A loop version of Horner's Rule can use:

$O(1)$ space

because it does not need recursive stack frames.

6. Day 3: Fibonacci Loop and Recursion

6.1 What Is Fibonacci?

Fibonacci is a sequence where each number is made by adding the previous two numbers.

The sequence starts like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Formula:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n - 1) + fib(n - 2)
```

Example:

```
fib(5) = fib(4) + fib(3)
```

6.2 Fibonacci Values

n	fib(n)
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13

6.3 Fibonacci Using a Loop

The loop version is usually efficient and beginner-friendly.

```
#include <stdio.h>

int fibLoop(int n) {
    if (n <= 1) {
        return n;
    }

    int a = 0;
    int b = 1;
    int c;

    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }

    return b;
}
```

```
}  
  
int main() {  
    printf("%d\n", fibLoop(6));  
    return 0;  
}
```

Output:

8

6.4 How the Loop Version Works

For `fibLoop(6)`:

Start:

```
a = 0  
b = 1
```

Now update:

```
i = 2: c = 0 + 1 = 1, a = 1, b = 1  
i = 3: c = 1 + 1 = 2, a = 1, b = 2  
i = 4: c = 1 + 2 = 3, a = 2, b = 3  
i = 5: c = 2 + 3 = 5, a = 3, b = 5  
i = 6: c = 3 + 5 = 8, a = 5, b = 8
```

Final answer:

8

6.5 Basic Recursive Fibonacci Code

```
#include <stdio.h>  
  
int fibRecursive(int n) {
```

```
    if (n <= 1) {
        return n;
    }

    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

int main() {
    printf("%d\n", fibRecursive(6));
    return 0;
}
```

Output:

8

This code looks clean, but it is inefficient.

6.6 Base Case in Fibonacci

The base case is:

```
if (n <= 1) {
    return n;
}
```

This covers two cases:

```
fib(0) = 0
fib(1) = 1
```

6.7 Recursive Case in Fibonacci

The recursive case is:

```
return fibRecursive(n - 1) + fibRecursive(n - 2);
```

This means:

To find $\text{fib}(n)$, find $\text{fib}(n - 1)$ and $\text{fib}(n - 2)$, then add them.

6.8 Dry Run of `fibRecursive(5)`

Function call:

`fib(5)`

Expand it:

`fib(5) = fib(4) + fib(3)`

Now expand `fib(4)`:

`fib(4) = fib(3) + fib(2)`

Now expand `fib(3)`:

`fib(3) = fib(2) + fib(1)`

A larger view:

```
fib(5)
├─ fib(4)
│  ├─ fib(3)
│  │  ├─ fib(2)
│  │  │  ├─ fib(1)
│  │  │  └─ fib(0)
│  │  └─ fib(1)
│  └─ fib(2)
│     ├─ fib(1)
│     └─ fib(0)
└─ fib(3)
   ├─ fib(2)
   │  ├─ fib(1)
   │  └─ fib(0)
   └─ fib(1)
```

Final result:

```
fib(5) = 5
```

6.9 Why Basic Recursive Fibonacci Is Slow

Basic recursive Fibonacci is slow because it repeats the same work many times.

In the call tree for `fib(5)`, notice:

```
fib(3) appears more than once
fib(2) appears many times
fib(1) appears many times
fib(0) appears many times
```

The function does not remember earlier answers.

So it calculates the same values again and again.

This is called repeated work.

Simple warning:

Basic Fibonacci is a trap. It looks short, but it is inefficient.

6.10 Complexity of Fibonacci Loop vs Recursive Fibonacci

Version	Time Complexity	Space Complexity	Reason
Fibonacci loop	<code>O(n)</code>	<code>O(1)</code>	One loop, constant variables
Basic recursive Fibonacci	<code>O(2^n)</code>	<code>O(n)</code>	Many repeated recursive calls

The recursive Fibonacci function has `O(n)` space because the deepest chain is from `n` down to `0`.

But its time is much worse because the total number of calls grows very fast.

7. Day 4: Fibonacci Memoization

7.1 What Is Memoization?

Memoization means storing an answer after calculating it once.

Then, if the same answer is needed again, we reuse the stored answer.

Simple idea:

```
Do not solve the same problem twice.
```

Example:

```
If fib(4) is already calculated, store it.  
Next time fib(4) is needed, return the stored value immediately.
```

7.2 Why Fibonacci Needs Memoization

Basic recursive Fibonacci repeats values.

Example:

```
fib(5) needs fib(4) and fib(3)  
fib(4) also needs fib(3)
```

So `fib(3)` is calculated more than once.

Memoization fixes this by storing values in an array.

7.3 Memoization Array

We can create an array like this:

```
int F[100];
```

This array stores Fibonacci answers.

Example:

```
F[0] stores fib(0)
F[1] stores fib(1)
F[2] stores fib(2)
F[3] stores fib(3)
```

7.4 Why Use -1?

Before calculating, we fill the array with -1.

```
F[i] = -1;
```

Here:

```
-1 means not calculated yet
```

This works because Fibonacci values are never negative.

Valid Fibonacci values are:

```
0, 1, 1, 2, 3, 5, 8, ...
```

So -1 is a safe marker for an unknown value.

7.5 Memoized Fibonacci Code

```
#include <stdio.h>

int F[100];

void initialize() {
    for (int i = 0; i < 100; i++) {
        F[i] = -1;
    }
}

int fibMemo(int n) {
```

```
    if (n <= 1) {
        F[n] = n;
        return n;
    }

    if (F[n] != -1) {
        return F[n];
    }

    F[n] = fibMemo(n - 1) + fibMemo(n - 2);
    return F[n];
}

int main() {
    initialize();

    printf("%d\n", fibMemo(6));

    return 0;
}
```

Output:

8

7.6 How Memoized Fibonacci Works

Suppose we call:

fibMemo(6)

The function checks:

```
if (F[n] != -1)
```

This means:

If the answer is already stored, return it immediately.

If the value is not stored, it calculates it:

```
F[n] = fibMemo(n - 1) + fibMemo(n - 2);
```

Then it saves the answer in `F[n]`.

7.7 Example of Saved Values

After calculating `fibMemo(6)`, the array may contain:

Index	Stored Value
<code>F[0]</code>	0
<code>F[1]</code>	1
<code>F[2]</code>	1
<code>F[3]</code>	2
<code>F[4]</code>	3
<code>F[5]</code>	5
<code>F[6]</code>	8

Now if the function needs `fibMemo(4)` again, it can directly return:

`F[4]`

It does not need to calculate it again.

7.8 Complexity of Memoized Fibonacci

Version	Time Complexity	Space Complexity	Reason
Basic recursive Fibonacci	<code>O(2^n)</code>	<code>O(n)</code>	Repeats the same values many times
Memoized Fibonacci	<code>O(n)</code>	<code>O(n)</code>	Each value is calculated once and stored

Memoization improves time from exponential to linear.

That is a major improvement.

7.9 Memoization vs Normal Recursion

Normal Recursion	Memoized Recursion
Calculates values again and again	Stores values after calculating
Can be very slow	Usually much faster
Uses only stack memory	Uses stack plus array memory
Simple but inefficient for repeated subproblems	Better for repeated subproblems

8. Day 5: nCr and Pascal Recursion

8.1 What Is nCr ?

nCr means combinations.

It answers this question:

From n items, how many ways can we choose r items?

Example:

$5C2$

means:

From 5 items, choose 2 items.

The answer is:

10

8.2 nCr Formula

The formula is:

$$nC_r = n! / (r! * (n - r)!)$$

Example:

$$5C_2 = 5! / (2! * (5 - 2)!)$$

So:

$$5C_2 = 5! / (2! * 3!)$$

Now calculate:

$$\begin{aligned} 5! &= 120 \\ 2! &= 2 \\ 3! &= 6 \end{aligned}$$

So:

$$\begin{aligned} 5C_2 &= 120 / (2 * 6) \\ 5C_2 &= 120 / 12 \\ 5C_2 &= 10 \end{aligned}$$

8.3 nCr Using Factorial Code

```
#include <stdio.h>

int factorial(int n) {
    if (n == 0) {
        return 1;
    }

    return factorial(n - 1) * n;
}

int nCrFormula(int n, int r) {
    if (r < 0 || r > n) {
        return 0;
    }
}
```



```
        return factorial(n) / (factorial(r) * factorial(n - r));
    }

    int main() {
        printf("%d\n", nCrFormula(5, 2));
        return 0;
    }
```

Output:

10

8.4 Warning About Factorial Formula

The factorial formula is easy to understand, but factorial values grow very fast.

Example:

```
5! = 120
10! = 3,628,800
20! is very large
```

So for larger values, normal `int` may overflow.

Simple warning:

The factorial formula is simple, but it is not always safe for large inputs.

8.5 Pascal's Triangle Idea

Pascal's Triangle is another way to calculate combinations.

Important formula:

```
nCr = (n - 1)C(r - 1) + (n - 1)Cr
```

Base cases:

```
nC0 = 1  
nCn = 1
```

Why?

There is exactly one way to choose nothing:

```
nC0 = 1
```

There is exactly one way to choose everything:

```
nCn = 1
```

8.6 Example of Pascal Recurrence

Find:

```
5C2
```

Using Pascal's formula:

```
5C2 = 4C1 + 4C2
```

Now:

```
4C1 = 3C0 + 3C1  
4C2 = 3C1 + 3C2
```

The problem breaks into smaller combination problems.

8.7 nCr Using Pascal Recursion Code

```
#include <stdio.h>  
  
int nCrPascal(int n, int r) {  
    if (r < 0 || r > n) {
```

```

        return 0;
    }

    if (r == 0 || r == n) {
        return 1;
    }

    return nCrPascal(n - 1, r - 1) + nCrPascal(n - 1, r);
}

int main() {
    printf("%d\n", nCrPascal(5, 2));
    return 0;
}

```

Output:

10

8.8 Base Case in Pascal Recursion

The base case is:

```

if (r == 0 || r == n) {
    return 1;
}

```

This represents:

```

nC0 = 1
nCn = 1

```

8.9 Recursive Case in Pascal Recursion

The recursive case is:

```

return nCrPascal(n - 1, r - 1) + nCrPascal(n - 1, r);

```

This means:

```
nCr = choosing current item + not choosing current item
```

In simpler terms:

```
Break the combination problem into two smaller combination problems.
```

8.10 Why Pascal nCr Is Tree Recursion

This line has two recursive calls:

```
return nCrPascal(n - 1, r - 1) + nCrPascal(n - 1, r);
```

Because the function calls itself more than once, it is tree recursion.

Tree recursion can repeat work.

Example:

```
5C2 = 4C1 + 4C2
```

Then:

```
4C2 = 3C1 + 3C2
```

Some subproblems may appear again in different branches.

So Pascal recursion is elegant, but it can become slow for larger inputs.

8.11 Factorial Formula vs Pascal Recursion

Method	Idea	Advantage	Risk
Factorial formula	Use <code>n! / (r! * (n-r)!)</code>	Easy to understand	Factorials can become very large

Method	Idea	Advantage	Risk
Pascal recursion	Use $nCr = (n-1)C(r-1) + (n-1)Cr$	Shows recursive structure clearly	Tree recursion can repeat work

8.12 Can nCr Also Use Memoization?

Yes.

Pascal recursion has repeated subproblems.

That means memoization can improve it.

The idea is similar to Fibonacci memoization:

Store already calculated nCr values.
Reuse them when needed again.

For example, if $3C1$ is calculated once, store it.

Next time $3C1$ appears, return the stored value.

9. Main Week 5 Concept: Repeated Work

The most important concept in Week 5 is repeated work.

Repeated work means the function solves the same smaller problem more than once.

Examples:

Problem	Repeated Work
Fibonacci recursion	$\text{fib}(3)$, $\text{fib}(2)$, $\text{fib}(1)$ are calculated many times
Pascal nCr recursion	Same combination values can appear in different branches
Basic Taylor thinking	Powers and factorials may be recalculated unnecessarily

Repeated work makes programs slow.

Memoization and better formulas reduce repeated work.

10. Main Week 5 Concept: Memoization

Memoization is a technique for improving recursive solutions.

It means:

Calculate once, store the answer, reuse it later.

Memoization is useful when:

- The recursive function has repeated subproblems
- The same input appears again and again
- The answer for the same input does not change

Good examples:

- Fibonacci
- Pascal `nCr`

Not every recursion needs memoization.

For example, simple factorial does not repeat the same subproblem.

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

Each value appears once.

So memoization is not very useful there.

11. Complete Week 5 Practice Program

```
#include <stdio.h>

int F[100];

void initializeFibArray() {
```

```

    for (int i = 0; i < 100; i++) {
        F[i] = -1;
    }
}

double taylor(int x, int n) {
    static double power = 1;
    static double fact = 1;

    if (n == 0) {
        return 1;
    }

    double result = taylor(x, n - 1);

    power = power * x;
    fact = fact * n;

    return result + power / fact;
}

double hornerHelper(double x, int n, double sum) {
    if (n == 0) {
        return sum;
    }

    sum = 1 + (x * sum) / n;
    return hornerHelper(x, n - 1, sum);
}

double horner(double x, int n) {
    return hornerHelper(x, n, 1);
}

int fibLoop(int n) {
    if (n <= 1) {
        return n;
    }

    int a = 0;
    int b = 1;
    int c;

    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
}

```

```

        return b;
    }

    int fibRecursive(int n) {
        if (n <= 1) {
            return n;
        }

        return fibRecursive(n - 1) + fibRecursive(n - 2);
    }

    int fibMemo(int n) {
        if (n <= 1) {
            F[n] = n;
            return n;
        }

        if (F[n] != -1) {
            return F[n];
        }

        F[n] = fibMemo(n - 1) + fibMemo(n - 2);
        return F[n];
    }

    int factorial(int n) {
        if (n == 0) {
            return 1;
        }

        return factorial(n - 1) * n;
    }

    int nCrFormula(int n, int r) {
        if (r < 0 || r > n) {
            return 0;
        }

        return factorial(n) / (factorial(r) * factorial(n - r));
    }

    int nCrPascal(int n, int r) {
        if (r < 0 || r > n) {
            return 0;
        }

        if (r == 0 || r == n) {

```



```

        return 1;
    }

    return nCrPascal(n - 1, r - 1) + nCrPascal(n - 1, r);
}

int main() {
    initializeFibArray();

    printf("Taylor e^1 up to 3 terms = %lf\n", taylor(1, 3));
    printf("Horner e^1 up to 3 terms = %lf\n", horner(1, 3));
    printf("Fibonacci loop fib(6) = %d\n", fibLoop(6));
    printf("Fibonacci recursion fib(6) = %d\n", fibRecursive(6));
    printf("Fibonacci memoization fib(6) = %d\n", fibMemo(6));
    printf("nCr formula 5C2 = %d\n", nCrFormula(5, 2));
    printf("nCr Pascal 5C2 = %d\n", nCrPascal(5, 2));

    return 0;
}

```

Expected output:

```

Taylor e^1 up to 3 terms = 2.666667
Horner e^1 up to 3 terms = 2.666667
Fibonacci loop fib(6) = 8
Fibonacci recursion fib(6) = 8
Fibonacci memoization fib(6) = 8
nCr formula 5C2 = 10
nCr Pascal 5C2 = 10

```

12. Week 5 Summary Table

Topic	Main Idea	Risk	Improvement
Taylor series	Build <code>e^x</code> term by term	Repeated power and factorial work	Horner's Rule
Horner's Rule	Rearrange expression into nested form	Formula is less obvious at first	Fewer multiplications
Fibonacci loop	Use variables to build sequence	Less connected to recursion practice	Efficient memory usage

Topic	Main Idea	Risk	Improvement
Basic Fibonacci recursion	<code>fib(n) = fib(n-1) + fib(n-2)</code>	Repeated work, exponential time	Memoization
Fibonacci memoization	Store Fibonacci values in array	Needs extra array	Reduces time to <code>O(n)</code>
nCr formula	Use factorial formula	Factorial overflow for large inputs	Useful for small inputs
nCr Pascal recursion	Use Pascal recurrence	Tree recursion repeats work	Memoization can help

13. Complexity Summary

Function	Time Complexity	Space Complexity	Reason
Taylor recursion	<code>O(n)</code>	<code>O(n)</code>	One recursive call per term
Horner recursion	<code>O(n)</code>	<code>O(n)</code>	One recursive call per term
Fibonacci loop	<code>O(n)</code>	<code>O(1)</code>	Loop uses fixed variables
Basic Fibonacci recursion	<code>O(2^n)</code>	<code>O(n)</code>	Tree recursion repeats work
Fibonacci memoization	<code>O(n)</code>	<code>O(n)</code>	Each value is calculated once
nCr formula	Usually <code>O(n)</code>	<code>O(n)</code> with recursive factorial	Factorial calls use recursion
nCr Pascal recursion	Exponential	<code>O(n)</code>	Tree recursion creates repeated branches

14. Common Beginner Mistakes in Week 5

Mistake 1: Thinking Short Code Is Always Better

Basic Fibonacci code is short:

```
return fib(n - 1) + fib(n - 2);
```

But it is slow because it repeats work.

Short code is not always efficient code.

Mistake 2: Forgetting to Initialize the Memo Array

Wrong:

```
int F[100];  
printf("%d", fibMemo(6));
```

Problem:

The array may contain garbage values.

Correct:

```
initialize();  
printf("%d", fibMemo(6));
```

Mistake 3: Not Understanding **-1** in Memoization

Wrong thinking:

```
-1 is part of the Fibonacci sequence.
```

Correct thinking:

```
-1 means this value has not been calculated yet.
```

Mistake 4: Thinking Pascal nCr Is Always Fast

Pascal recursion looks clean:

```
return nCrPascal(n - 1, r - 1) + nCrPascal(n - 1, r);
```

But it makes two recursive calls.

So it is tree recursion and can become slow.

Mistake 5: Ignoring Integer Overflow in Factorial nCr

Factorials grow quickly.

Example:

20! is already very large

Using `int` may overflow.

So the factorial formula is fine for small values, but not safe for large values.

Mistake 6: Calling Taylor Function Multiple Times Without Resetting Static Variables

The Taylor function uses:

```
static double power = 1;  
static double fact = 1;
```

Static variables remember old values.

So repeated calls can cause wrong results if not handled carefully.

15. How to Trace Week 5 Problems

Use this method:

1. Write the function call.
2. Identify the base case.
3. Identify the recursive case.
4. Check how many recursive calls are made.
5. If there is more than one recursive call, draw a tree.
6. Look for repeated subproblems.
7. Ask whether memoization can help.

8. Count approximate function calls.
 9. Write time complexity.
 10. Write space complexity.
-

16. Practice Questions

Question 1

What is the output?

```
printf("%d", fibLoop(6));
```

Answer:

```
8
```

Reason:

```
0, 1, 1, 2, 3, 5, 8
```

So `fib(6) = 8`.

Question 2

What is the base case of Fibonacci recursion?

Answer:

```
if (n <= 1) {  
    return n;  
}
```

Reason:

```
fib(0) = 0  
fib(1) = 1
```

Question 3

Why is basic Fibonacci recursion slow?

Answer:

Because it calculates the same values again and again.

Example:

`fib(3)` and `fib(2)` appear multiple times in the call tree.

Question 4

What does memoization mean?

Answer:

Memoization means storing an answer after calculating it once, then reusing it later.

Question 5

Why do we initialize the Fibonacci memo array with `-1`?

Answer:

Because `-1` means the value has not been calculated yet.

Fibonacci values are not negative, so `-1` is safe as an unknown marker.

Question 6

What is the formula for `nCr`?

Answer:

$$nCr = n! / (r! * (n - r)!)$$

Question 7

What is Pascal's recurrence for `nCr`?

Answer:

$$nCr = (n - 1)C(r - 1) + (n - 1)Cr$$

Question 8

Why is Pascal `nCr` tree recursion?

Answer:

Because it makes two recursive calls:

```
nCrPascal(n - 1, r - 1)
nCrPascal(n - 1, r)
```

Question 9

What is the main benefit of Horner's Rule?

Answer:

It reduces unnecessary multiplication by rewriting the expression in nested form.

Question 10

Which is faster: basic Fibonacci recursion or memoized Fibonacci?

Answer:

Memoized Fibonacci is faster.

Basic Fibonacci recursion has exponential time.

Memoized Fibonacci has linear time.

17. Week 5 Mastery Checklist

Before moving to Week 6, you should be able to do these:

- Explain Taylor series in simple words
- Write the Taylor series formula for `e^x`
- Explain what `power`, `fact`, and `sum` mean
- Trace `taylor(1, 3)` manually
- Explain why static variables can be risky
- Explain Horner's Rule in simple words
- Trace `horner(1, 3)` manually
- Explain the Fibonacci sequence
- Write Fibonacci using a loop
- Write Fibonacci using basic recursion
- Draw the call tree for `fib(5)`
- Explain why basic Fibonacci recursion is slow
- Explain repeated work
- Explain memoization
- Write Fibonacci using memoization
- Explain why `-1` means unknown in memoization
- Compare Fibonacci loop, recursion, and memoization
- Explain what `nCr` means
- Write `nCr` using the factorial formula
- Explain Pascal's Triangle recurrence
- Write `nCr` using Pascal recursion
- Explain why Pascal recursion is tree recursion
- Identify which Week 5 problems repeat work
- Explain when memoization is useful

18. Final Week 5 Summary

Week 5 teaches advanced recursive problem solving.

The main topics are:

Taylor series
Horner's Rule
Fibonacci
Memoization
nCr
Pascal recursion

The biggest lesson is:

```
Correct recursion is not always efficient recursion.
```

Basic Fibonacci is the best example.

This version works:

```
return fib(n - 1) + fib(n - 2);
```

But it repeats work many times.

Memoization improves it by storing values after calculating them once.

Pascal `nCr` also shows tree recursion because each function call creates two more recursive calls.

Taylor series shows how recursion can build a mathematical series term by term.

Horner's Rule shows how rearranging a formula can reduce unnecessary multiplication.

The main Week 5 rule is:

When recursion repeats the same smaller problems, think about memoization.

19. Final Rule for Week 5

Every time you see an advanced recursive function, ask these questions:

1. What is the base case?
2. What is the recursive case?
3. How many recursive calls are made?
4. Is this linear recursion or tree recursion?
5. Are the same subproblems repeated?
6. Can memoization help?
7. What is the time complexity?
8. What is the space complexity?
9. Is there a faster formula or rearrangement?
10. Is the recursive version better than a loop here?

If you can answer these questions, you are not just writing recursion. You are analyzing recursion.